

Trabalho Prático: Simulador de Livro de Ofertas e Performance de Estruturas

Disciplina: Estrutura de Dados em Python
Prof. Marcos Mansano Furlan

Semestre 2026

1 Objetivo

Este trabalho tem como objetivo aplicar conceitos de estruturas de dados lineares (Listas Encadeadas, Pilhas e Filas) no desenvolvimento de um motor de negociação financeira. O foco será a análise assintótica e a comparação prática da performance dessas estruturas em cenários de grande volume de dados, preparando a base conceitual para o estudo futuro de estruturas não-lineares.

2 Dinâmica de Grupo e Repositório

- O trabalho será feito em grupos de 4 ou 5 pessoas.
- **Uso do GitHub:** É obrigatória a criação de um repositório para o projeto. O histórico de *commits* servirá para validar a evolução incremental do código e a participação individual.
- O link do repositório deve constar no relatório e no formulário de entrega no e-disciplinas.

3 O Problema: Simulador de Livro de Ofertas (*Order Book*)

O sistema deve gerenciar ordens de compra e venda de ativos de forma eficiente. O desafio central é o “casamento” (*match*) de ordens: quando um comprador aceita pagar o preço que um vendedor está pedindo (ou mais), uma transação deve ser gerada.

3.1 Entidades e Atributos

Cada ordem de negociação inserida no sistema é composta por:

- **ID** (*int*): Identificador único da ordem.
- **Tipo** (*char*): ‘C’ para Compra ou ‘V’ para Venda.

- **Preço** (*float*): Valor unitário que o investidor aceita pagar ou receber.
- **Quantidade** (*int*): Volume de ações a serem negociadas.
- **Timestamp** (*t*): Momento exato da entrada da ordem no sistema.

3.2 Estruturas de Dados e Fluxo

O sistema deve operar com três estruturas principais:

1. **Fila de Entrada (Fila/Queue):** Todas as novas ordens entram em uma fila FIFO para aguardar o processamento pelo motor.
2. **Livro de Ofertas (Listas Duplamente Encadeadas Ordenadas):**
 - **Lista de Compras:** Mantida em ordem **decrecente** de preço (melhor comprador no início).
 - **Lista de Vendas:** Mantida em ordem **crescente** de preço (melhor vendedor no início).
3. **Sistema de Undo (Pilha/Stack):** Armazena o ID das ordens inseridas com sucesso no livro para permitir o cancelamento rápido da última ação.

4 Requisitos Técnicos (Implementação)

O grupo deve implementar as estruturas de dados “do zero”, sem o auxílio de módulos integrados ou tipos nativos como `list` (para simular a lógica de ponteiros/referências).

- **Gestão de Listas Encadeadas:** Inserção ordenada percorrendo os nós ($O(n)$) e remoção de qualquer ponto da lista (religamento de ponteiros).
- **Interface de Fila e Pilha:** Implementação baseada em nós para garantir $O(1)$ nas operações de extremidade (`push/pop`, `enqueue/dequeue`).
- **Motor de Match:** Sempre que uma nova ordem entrar, o sistema deve verificar se o preço do início da lista de compra é $\geq$ ao início da lista de venda. Se sim, executa a transação e remove ou atualiza os nós.
- **Análise de Desempenho:** O uso de estruturas lineares para busca e inserção ordenada ($O(n)$) deve ser documentado para posterior comparação com árvores.

5 Critérios de Avaliação

5.1 Implementação (NI - 6.0 pontos)

- **Lógica de Ponteiros:** Manipulação correta de referências (*next/prev*) e ausência de nós órfãos.
- **Orientação a Objetos:** Organização em classes (`Node`, `LinkedList`, `Stack`, `Queue`).
- **Versionamento:** Avaliação da consistência dos *commits* no GitHub.

5.2 Relatório de Performance (NR - 4.0 pontos)

- **Complexidade Teórica:** Análise do custo $O(n)$ para inserção/busca em listas encadeadas.
- **Análise Empírica:** Gráficos gerados via Jupyter Notebook comparando o tempo de processamento conforme o volume de ordens cresce.

5.3 Defesa Oral e Avaliação de Pares

A nota individual será ajustada pelo multiplicador de defesa oral (NA) e avaliação de pares ($NAAP$).

6 Composição da Nota Final

$$NT = (NI + NR) \times \min(NA, NAAP)$$

7 Instruções de Entrega

Entregar arquivo compactado contendo o código (.py), o notebook (.ipynb) e o link do repositório GitHub.

IMPORTANTE

Não é permitido o uso de estruturas prontas (como listas nativas do Python ou `collections.deque`). Todas as estruturas devem ser construídas manualmente através do encadeamento de nós.